INSTITUT D’ ENSEIGNEMENT SUPÉRIEUR DE RUHENGERI

ARTIFICIAL INTELLIGENCE — INDIVIDUAL ASSIGNMENT 2

Musanze Harvest Intelligence Service

An Individual Reproducible Machine Learning System and API

Student Name: Hirwa Honorine
Registration Number: 25/28159
Institutional Username: ug2528159@ines.ac.rw
Verification Code: AI-A2-8159
Random Seed: 8159
Submission Date: 19 October 2026

1. Introduction

Potato farming is an important agricultural activity in the Musanze area. A cooperative handling farm records may need to estimate expected potato yield, identify farms that may have high production risk, and understand similarities between different fields.

The Musanze Harvest Intelligence Service is a Python-based machine-learning prototype developed to demonstrate how data analysis can support these tasks. It generates a reproducible synthetic dataset, validates and preprocesses the data, trains machine-learning models, evaluates their performance, and provides predictions through a command-line interface and a FastAPI service.

2. Objectives

The objectives of this assignment are to:

Generate a synthetic dataset using the assigned student seed.Validate and clean data, including missing values and duplicate records.Compare regression models for yield estimation.Compare classification models for high-risk prediction.Group similar field records using clustering.Save model artifacts and provide reusable predictions through a CLI and API.Test the system and document its limitations and responsible use.

3. Intended Stakeholder

The intended stakeholder is a field officer or cooperative staff member who needs to examine farm conditions and review model-generated estimates. The system is a classroom prototype and does not replace agricultural experts or actual field inspections.

Scope: All generated records are synthetic. They are not survey results collected from real farmers in Musanze.

4. Dataset Generation and Reproducibility

4.1 Dataset generation

The dataset is generated by data/generate_data.py using Python and NumPy's default_rng with the assigned seed of 8159. The base row count is calculated as:

Base rows = 240 + (8159 mod 41) = 240

The generator creates synthetic field conditions within the ranges specified in the assignment. Numeric values are saved to four decimal places. The dataset is regenerated by running `run_all.py`, allowing the same seed and code to reproduce the data.

4.2 Data dictionary

Feature	Description
rainfall_mm	Rainfall in millimetres
avg_temp_c	Average temperature in degrees Celsius
soil_moisture_pct	Soil moisture percentage
fertilizer_kg	Fertilizer quantity in kilograms
field_size_ha	Field area in hectares
season	Season category A or B
cooperative_support	Cooperative support indicator, 0 or 1
yield_tons	Synthetic yield target for regression
high_risk	Binary target derived from yield relative to the dataset median

4.3 Yield Generation

Synthetic yield is calculated using rainfall, soil moisture, fertilizer, field size, temperature, cooperative support, season effects, and random noise.

The formula assigns different weights to these factors. Season A adds 0.50, while Season B subtracts 0.20. Random noise follows a normal distribution with a mean of 0 and a standard deviation of 0.80. The final yield is limited to a minimum of 0.50 tons.

The `high_risk` label is assigned when yield per hectare is below the dataset median. This label supports classification experiments but does not represent a verified agricultural risk assessment.

4.4 Data quality and fingerprint

The dataset contains 240 base rows, with three duplicate rows added intentionally. The pipeline removes duplicates and checks missing values before modelling.

- **Base rows:** 240
- **Rows after adding duplicates:** 243
- **Rows after duplicate removal:** 240
- **Missing soil-moisture values:** 13
- **Missing fertilizer values:** 13
- **Dataset SHA-256:** `edd166f4a2b1c06b261363a6ba40f8140ece2c7fe50bfbcca2a4ecc139ad9e7d`

This fingerprint was produced by the latest successful pipeline run. It must match artifacts/data_fingerprint.txt and the fingerprint in the final report.

5. Pipeline Design and Preprocessing

5.1 System workflow

The system follows a reproducible pipeline:

1. Generate the synthetic dataset using the assigned seed.
2. Validate the schema, ranges, missing values and duplicates.
3. Split the data into training and testing sets.

4. Fit preprocessing transformations using training data only.
5. Train baseline and candidate regression and classification models.
6. Evaluate the candidate models on held-out test data.
7. Scale the input features and compare clustering configurations.
8. Save the selected models, metrics, plots and predictions.
9. Load the saved artifacts for CLI and API inference.

5.2 Data preprocessing

Numeric missing values are handled using imputation. Categorical variables, such as `season`, are encoded for use by machine-learning algorithms. Numeric scaling is applied where required, particularly for clustering and models that benefit from standardized inputs.

The preprocessing transformations are fitted using training data only for supervised modelling. The same fitted transformations are reused during inference so that new records are processed consistently with the training records.

5.3 Training and testing

A fixed train/test split uses `random_state=8159`. The training set is used to fit models and preprocessing transformations, while the held-out test set is used to evaluate predictive performance.

The pipeline compares each candidate with a simple baseline. The regression baseline predicts the mean training target. The classification baseline predicts the majority class in the training data.

5.4 Preventing target leakage

Target leakage occurs when information that reveals the answer is incorrectly included among the model inputs. For classification, `yield_tons` and `high_risk` must not be used as input features. The yield target must also be excluded from clustering inputs.

Using only the appropriate input features helps ensure that evaluation reflects a realistic prediction task.

5.5 Software structure

The project separates data generation, preprocessing, model training, evaluation, prediction and API functionality into reusable Python modules. `run_all.py` orchestrates the pipeline, `predict.py` performs command-line inference, and `app.py` exposes the saved models through FastAPI.

This structure makes the system easier to test, maintain and reproduce.

6. Regression and Classification Results

6.1 Regression: yield estimation

Regression predicts a numerical value. In this project, the target is **`yield_tons`**, generated from the synthetic field conditions.The pipeline compares a mean baseline with at least two regression models using held-out test data. The evaluation metrics are Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and R-squared (R^2).

- ✓ **MAE** measures the average absolute difference between predictions and actual synthetic targets.
- ✓ **RMSE** gives greater weight to larger prediction errors.
- ✓ **R^2** indicates how well the predictions explain variation in the held-out target values; it can be negative when performance is worse than the mean baseline.

Regression comparison

Model	MAE	RMSE	R^2
Mean baseline	2.324	2.839	-0.017
Regression candidate1	0.800	1.002	0.873
Regression candidate 2	0.981	1.205	0.817

Selected regression model: Linear Regression (`linear_regression`).

Selection reason: Linear Regression performed best, with the lowest RMSE (1.002) and MAE (0.800), and the highest R^2 (0.873). The residual plot helps identify patterns in prediction errors and areas where the model may not perform well.

One regression error: For record 213, the actual yield was 10.9212 tons, but the model predicted 13.6030 tons, overestimating by 2.6818 tons. This shows that the model can make errors when predicting individual records.

6.2 Classification: high-risk prediction

Classification predicts whether a synthetic record belongs to the `high_risk` class. The pipeline compares a majority-class baseline with at least two candidate classifiers.The evaluation uses precision, recall, F1-score, a confusion matrix and ROC-AUC when valid.

- **Precision** measures how many records predicted as high risk were actually labelled high risk.
- **Recall** measures how many actual high-risk records were correctly identified.
- **F1-score** balances precision and recall.
- **ROC-AUC** measures ranking discrimination when both classes and valid probability scores are available.

Classification comparison

Model	Precision	Recall	F1	ROC-AUC
Majority baseline	0.000	0.000	0.000	0.500
Classifier candidate	0.938	1.000	0.968	0.994
Classifier candidate	0.966	0.933	0.949	0.963

Selection reason: Logistic regression was selected because it had the highest F1 score (0.968) among the candidate classifiers.

One classification error: The selected model produced 2 false positives and 0 false negatives. This means two synthetic records labelled low risk were incorrectly flagged as high risk. A field officer should review these records rather than assume they are genuinely high risk.

7. Clustering and Interpretation

7.1 Purpose of clustering

Clustering is an unsupervised machine-learning task that groups records with similar input characteristics without using the target label. In this project, clustering is used to explore similarities between synthetic potato-field records.

The clustering process standardizes the selected input features so that measurements in different units do not dominate the distance calculation simply because of their scale.

7.2 Selecting the number of clusters

The pipeline evaluates K-means configurations for K values from 2 through 6. The silhouette score is used to compare the separation and cohesion of the resulting clusters. A higher score generally indicates better separation for the evaluated configuration, although the score alone does not establish real-world meaning.

Clustering comparison

K (clusters)	Silhouette score
2	0.1586
3	0.1351
4	0.1463
5	0.1428
6	0.1447

Selected cluster count: K = 2, according to the latest pipeline output.

Selection justification: K = 2 was selected because it had the highest silhouette score (0.1586) among the tested values K = 2 to 6. However, the low score indicates limited separation between clusters.

7.3 Cluster profiles

Cluster 0

Number of records: 120 records in Cluster 0 and 120 records in Cluster 1.

Observed characteristics: Cluster 0 has higher average soil moisture (52.478%), lower average fertilizer use (114.160 kg), and an average field size of 2.034 ha. Cluster 1 has lower average soil moisture (48.446%), higher average fertilizer use (131.144 kg), and an average field size of 2.068 ha. These patterns come from synthetic data and do not establish cause and effect.

Possible operational use: The groups may help a field officer identify records with similar conditions for further review and decide which records may need closer inspection. The clusters should not be treated as proof of actual farm conditions.

Cluster 1

Number of records: 120.

Observed characteristics: Lower average soil moisture (48.446%) and higher fertilizer use (131.144 kg).Possible operational use: Help the cooperative compare similar records and identify where more information is needed.

Cluster interpretation: Cluster 1 records had lower soil moisture and higher fertilizer use than Cluster 0. These patterns do not prove causes or indicate better or worse farms.The `clusters.png` plot visualizes the two clusters.

7.4 Operational value

The cluster profiles may help cooperative staff explore the diversity of field conditions and organize follow-up questions. They should be used only as exploratory information and must not be interpreted as proof of actual farm performance.

8. Prediction CLI and FastAPI Service

8.1 Command-line prediction

The **`predict.py`** script loads the saved preprocessing and model artifacts and accepts a JSON record containing the seven required input features.The output includes:Estimated yield in tons,High-risk class prediction,High-risk probability,Cluster label,Verification code **`AI-A2-8159`**.

A valid prediction can be run with:**`python predict.py --json samples/valid.json`** ,The script must reject missing or malformed input with a clear error rather than generating an unreliable prediction.

8.2 API endpoints

The FastAPI application in `app.py` provides three endpoints.

Method and path	Purpose
GET /health	Checks service status and returns the verification code.
GET /model-info	Returns identity, seed, fingerprint and selected model information.
POST /predict	Validates a field record and returns model predictions.

The local API can be started using:**`uvicorn app:app --host 127.0.0.1 --port 8000`**

Interactive API documentation is available at **`http://127.0.0.1:8000/docs.`**

8.3 Example prediction

The following example shows the response format. The numeric values must be copied from a current, successful prediction if presented as actual results.

Sample Prediction Output

Estimated yield: 11.3504 tons

High-risk prediction: 0 (not classified as high risk)

High-risk probability: 0.1337 (approximately 13.37%)

Cluster: 1

Verification code: AI-A2-8159

8.4 Automated testing

Automated tests are run using:**`python -m pytest -q`**

Automated Testing: The assignment requires at least six tests covering valid predictions, missing fields, incorrect data types, range validation, reproducibility and API responses.

Actual test result: 12 passed, 1 warning in 27.81s

8.5 Error handling

A test using `samples/invalid.json` returned:

`Prediction input error: Missing required feature(s): cooperative_support.` This demonstrates that the CLI detects an incomplete record and refuses to make a prediction from the missing required feature. API requests must similarly return useful client error responses for invalid input.

8.6 Reproducibility check

The pipeline was run twice using student ID **`ug2528159@ines.ac.rw`** and seed **`8159`**. The latest run produced fingerprint **`edd166f4a2b1c06b261363a6ba40f8140ece2c7fe50bfbcca2a4ecc139ad9e7d`**.The fingerprint matched the preceding run. This supports reproducibility of the generated dataset. Final verification must also confirm that the report and saved artifacts correspond to the same run.

9. Responsible AI, Limitations and Reflection

9.1 Intended use

The Musanze Harvest Intelligence Service is an educational prototype intended to demonstrate a reproducible machine-learning pipeline. It can illustrate how synthetic farm data may be used to estimate yield, predict a generated risk label, explore clusters and serve validated predictions through an API.

9.2 Limitations

The dataset is synthetic, so the model results may differ from real farm conditions. It does not include all factors affecting agriculture, such as pests, weather changes and soil differences. The high-risk label is based on generated yields, while clustering shows similarities in data rather than confirmed real-world farm categories.

9.3 Responsible use and human oversight

The system must not be used to make decisions about farmers' access to loans, insurance, public services or penalties. It must not replace an agronomist, cooperative officer or field inspection.

Any future deployment using real farm records would require representative data, permission and appropriate data governance, independent validation, monitoring of prediction errors, secure handling of records, and a qualified person who can review and reject model outputs.

9.4 Personal reflection

During this assignment, I learned to build a reproducible data pipeline, compare machine learning models, evaluate prediction errors, and use an API. I also learned that good model scores do not guarantee reliable real-world decisions, so data quality and limitations must always be considered.

My observed regression error: The largest absolute prediction error was 2.682 tons at test row 213. This means the model’s estimated yield differed from the actual synthetic yield by 2.682 tons.

My observed classification error: The logistic regression model produced 2 false positives and 0 false negatives. This means two low-risk synthetic records were incorrectly flagged as high risk.

My observed cluster: Cluster 0 contained 120 records, with average soil moisture of 52.478% and fertilizer use of 114.160 kg. Cluster 1 had lower average soil moisture and higher fertilizer use. These are patterns in synthetic data.

9.5 Conclusion

The Musanze Harvest Intelligence Service demonstrates an end-to-end machine-learning workflow: generating data, validating and preprocessing records, comparing and evaluating models, saving artifacts, making command-line predictions and serving requests through FastAPI.The system meets its educational purpose only when its output is reproducible, its limitations are clearly communicated, and its predictions are treated as decision-support information rather than verified agricultural facts.

Student: Hirwa Honorine
Registration Number: 25/28159
Verification Code: AI-A2-8159